How gcc-Linux-implementation works for code-injection-detection
https://github.com/code-injection-detection/gcc-Linux-Implementation
Dec-2017

This is an unofficial guide. Many things are temporary and about to change. Also, this is a high-level approach and we do not delve into the details.

Ok let's start:

Introduction

What the code tries to do:

First we need to talk about the general idea of this repo. Its goal is to emulate the secure system described by Lipton, Ostrovsky and Zikas here:
http://drops.dagstuhl.de/opus/frontdoor.php?source_opus=6311

This publication talks about a secure system that defends against an adversary who has arbitrary memory write capability. Specifically, we assume a remote attestator, who periodically queries the system which may be under attack, and verifies that the system has not (or has) been attacked. The power that is given to the adversary is a magic wand (let's say a radiation wand), that allows him to overwrite specific bits of the memory at any time.
Imagine he moves his wand, and the bit at address X is set to 1, and another bit at address Y is set to 0. Or something else. We must point out the fact that these changes are not done through the CPU instructions, but from an outside source (magic wand / radiation / row hammer attacks / etc).
The adversary has some restrictions however: He cannot overwrite something that is in the CPU (only in the memory), and he cannot read the memory before he can write to it. However, if he overwrites a place and me puts his own code there that outputs the rest of the memory, this way he can read it. But he needs to overwrite a place first.

The authors defeat such an adversary by the following algorithm:
They split the useful bytes of the application into blocks, and among them they interleave keyshare bytes (which together constitute a key) and MAC bytes (more details in the paper). So the memory structure is now

<useful bytes1><keyshares1><MAC bytes1> <useful bytes2><keyshares2><MAC bytes2> … etc …

(the keyshares are actually 2 groups of keyshare bytes, to avoid Time-of-check-time-

of-use attacks in the attetation procedure – more details about that in the paper).

The CPU, every time it fetches a block, verifies the corresponding MAC bytes. If they are wrong aborts the execution. Also, periodically, the remote attestator queries the system with a challenge. The system needs to gather all the keyshares for the memory to reply correctly. The attacker will either invalidate a specific MAC, or the key that is spread in the memory in the keyshares. The proof of correctness is in the paper, and we won't go into any more details – you should probably read that because the rest won't make much sense.

Here we talk about our implementation, which tries to emulate the performance of such a system.

We simplify a process' memory to have only 4 parts: text (code), data (globals), heap, stack.
We interleave keyshares + MAC bytes among the useful bytes in these 4 parts, and we use special functions (secure getters/setters) to access them.
For the code, we split the assemby into blocks, add NOP instructions (which will be then replaced by keyshare+MAC bytes), add a jmp above the NOPs to reach the next block (we don't want to execute the bytes that are keyshares/MACs), and, depending on the configuration, we add (in various parts inside the block) code that does MAC verification (more on that later).
For the globals, we organize them into a C struct. Among the actual variables, we add keyshares+MAC bytes into that struct too. We use special functions (getters/setters) to interact with the variables.
For the stack, we allocate an array in the normal, unsecured heap. We interleave keyashares + MACs, use getters/setters, and we implement our own calling convention. Instead of calling a function, we push its arguments into our "secure" stack, we push the return address, base pointer and local variables. Then we goto the start of the function. When we return, we goto the return address (and pop the stack frame).
For the heap, we allocate another array in the heap. Same story, keyshares+ MACs, getters/setters. We implement (using the secure globals and the secure stack) our own version of malloc/free, which keep two doubly linked lists, one for the allocated chunks and one for the free. Basically we copy the idea of dlmalloc (http://g.oswego.edu/dl/html/malloc.html) , and use it in our secure heap, without optimizations.

Great. We have described what the code generally does.

Now we will move on to talk about how each part is secured. We won't delve into our code just now, we will first describe the ideas of implementing.

Before we move on however, it would be a good idea if you tried to run our code.
Make sure you have the dependencies installed : A java compiler, gcc, libssl-dev

python3-pycparser binutils.
Now go to **code/** and run **./automate.sh** . It tells you it needs some parameters, and gives an example on how to execute it. Do it, and after a minute or so (for the subsequent runs it takes significantly less), it will produce an executable. You can run it (**./main_program_ksec**) and observe the output. Apart from the executable, you will see it has created a lot of other files – thy are mainly the copied templates because we can't touch them, but we can alter their copies.

How the various parts of the memory are secured

How the code is secured:

Now we are going to talk about how the code is changed, in more detail.
Let's assume that we have a given program that already initializes the secure heap and stack, and uses our secure convention of calling the functions, and the secure getters and setters in C language. That program must be compiled in a way that its code will be secure, too. We have not talked about the securing of such structures, but for now assume that that's how it is.

There are some parts that do not have to be secured, however. The functions that implement the mac initialization, verification or the secure getters/setters would normally be implemented in hardware. So we have them in separate files which are compiled with all optimizations enabled, and afterwards linked with the rest of the program.

In the end, we have a .c file (main_program.c in **code/** directory) which includes other .c files and .h files, and is linked with fully optimized object files. That compilation unit has all the secure code, and is being assembled into main_program.s .
So now we have all the secure code inside an assembly file, and it's time to insert the verification code, the keys and the macs. That is the job of **src/ Secure_Assembly_v2.java** (outside of **code/** directory) – you don't have to open it yet, which implements the following idea:

Consider some assembly code:

```
        ...
        movq    -24(%rbp), %rax
        cmpq    -40(%rbp), %rax
        jl      .L678
.L665:
        movq    -32(%rbp), %rax
```

```
        movq    %rax, -56(%rbp)
        movq    -56(%rbp), %rax
        addq    $88, %rsp
        ...
```

That code, as the scheme dictates, must have keyshares+MAC bytes interleaved among the actual bytes of the code. If we simply add the bytes, the execution environment will simply try to run these “random” bytes and will crash. That will not happen however, if we insert jumps above the extra bytes. Note: At first, we insert one-byte NOP commands for the keyshares+MACs, they will be replaced later however by th actual bytes.
There is also another problem. X86 instructions are variable size (typically 1-15 bytes long), but our scheme requires fixed size blocks, We solve this by padding extra NOPs after the jmp.
So, the code would be like that:

```
        …
        <previous_NOPs>

.BLCK_1500:
        movq    -24(%rbp), %rax
        cmpq    -40(%rbp), %rax
        jl      .L678
.L665:
        movq    -32(%rbp), %rax
        jmp .BLCK_1501

        <NOPs for padding>
        <NOPs for keyshares>
        <NOPs for MACs>

.BLCK_1501:
        movq    %rax, -56(%rbp)
        movq    -56(%rbp), %rax
        addq    $88, %rsp
        jmp .BLCK_1502:

        <NOPs>
        ...
```

There is one minor problem. Some commands (the jump command family) are not fixed size – their size depends on the offset (which changes depending on how many blocks away their target may be). We can enforce a 32-bit offset by adding a .d32 at the end of the opcode: jl .L678 → jl.d32 .L678
We also add, after each jump that goes to the next block, some code canary bytes, and some metadata that help us when verifying the MAC. Specifically, we add:
a) 3 canary bytes with value 0x42, to tell us “the jmp before me is a jmp above keyshares+MACs”
b) 2 bytes that denote the size of the code bytes, from the .BLCK_x label to (including) the jmp .BLCK_(x+1).

c) 2 bytes that denote the number of padded NOPs.

So the code becomes:

```
        …
        <previous_NOPs>

.BLCK_1500:
        movq    -24(%rbp), %rax
        cmpq    -40(%rbp), %rax
        jl.d32   .L678
.L665:
        movq    -32(%rbp), %rax
        jmp.d32 .BLCK_1501
        <canary bytes>
        <metadata bytes>
        <NOPs for padding>
        <NOPs for keyshares>
        <NOPs for MACs>

.BLCK_1501:
        movq    %rax, -56(%rbp)
        movq    -56(%rbp), %rax
        addq    $88, %rsp
        jmp.d32 .BLCK_1502:
        <canary bytes>
        <metadata bytes>
        <NOPs>
        ...
```

There is one major problem that remains however. While the hardware is supposed to know where each block resides and run the MAC verification every time it is accessed, we only have our emulation code that is executed – no extra MAC verifications. So, we must also add verification code whenever we believe that the code block is accessed.
Our verification code should do the following:
a) Save the current execution state (registers)
b) Check if we need MAC verification (we may not need if the block is in the code cache inside the CPU, we only need if it is not and it is added to the cache)
c) If yes, verify the MAC. If not, do nothing
d) Restore the current state

We have a function that does the checks and verification, do_verify_code_on_the_fly(). It emulates what the secure hardware would do. Its call consists of 3 assembly commands:

```
        pushfq
        call do_verify_code_on_the_fly
        popfq
```

(the push/pop f commands are there because the first command of the function is a

sub, which clobbers the flags register)
That code can be put practically anywhere, since it does not necessarily verify the MAC, but checks if it needs to be done. So, we put it after every suspicious point that may trigger a MAC verification:
a) After function starts
b) After call commands (except for the functions that emulate functionality done in hardware)
c) After labels
d) At block starts
e) We also place the code BEFORE a jump to register, and we check if that jump will land us in another block, and if yes, we verify it too.

That's how the code becomes:

```
        …
        <previous_NOPs>

.BLCK_1500:
        pushfq #start of block
        call do_verify_code_on_the_fly
        popfq
        movq    -24(%rbp), %rax
        cmpq    -40(%rbp), %rax
        jl.d32    .L678
.L665:
        pushfq #after label
        call do_verify_code_on_the_fly
        popfq
        movq    -32(%rbp), %rax
        jmp.d32 .BLCK_1501
        <canary bytes>
        <metadata bytes>
        <NOPs for padding>
        <NOPs for keyshares>
        <NOPs for MACs>

.BLCK_1501:
        pushfq #start of block
        call do_verify_code_on_the_fly
        popfq
        movq    %rax, -56(%rbp)
        movq    -56(%rbp), %rax
        addq     $88, %rsp
        jmp.d32 .BLCK_1502:
        <canary bytes>
        <metadata bytes>
        <NOPs>
        ...
```

All this added code incurs a heavy performance penalty. For everything except the verification, that's more or less the best we can do, because we emulate the system in software. Of course, specialized hardware wouldn't even have the jumps, it would

know where each block ended and it would jump over the keyshares and MACs automatically.
For the verification code however, there is something we can do: We can only count the NUMBER of the MAC verifications. After the execution, we can calculate the time it takes for one MAC verification, and this way find how much time all the MACs took. We can then add the time it takes for the code WITHOUT including the MAC verification code, and extract the final time this way.

So now we have an assembly file (main_program_sec.s) which has blocks of useful code, on-the-fly verification, jmps and padded nops. The next step is to replace the NOPs for keyshares+MACs with the proper bytes.
We assemble that assembly file into main_program_sec (an executable), and we iterate on it to replace the bytes.
That is the job of **src/Secure_Machine_Code_v2.java** , which runs through the main_program_sec, finds the jmps which have X nops after them, and replaces the keyshare bytes and the MAC bytes.
It also generates the keys for the secure stack and heap, and writes them to a file so that they can be initialized by the program when it runs. After that procedure the program (main_program_ksec) is ready to run.

How the globals are secured:

The secure global variables all reside in a struct. That struct is declared in **template_files/headers_needed_template.h .** After each useful variable, we pad with useless bytes to reach a full block (we don't group many grobal variables together in one block), and we also add keyshare and mac bytes. After the compilation (run of code/automate.sh), we can see that the code/main_program.c file has the initialization of a big global struct.
We access the globals using special getters/setters which take into account the fact that the MAC has to be verified/updated.
The way we do this right now, every block of <useful_bytes>||<keys>||<macs> can only hold only one global variable. Should someone want to use arrays of variables, he has to use the heap. In the heap, he can allocate the array he wants.
Here's the start of the global struct:

```
typedef struct global_variables_struct
{
//ATTENTION: GLOBAL VARIABLE FOLLOWING! | SIZE:double
double global_double_variable_for_testing;
//ATTENTION: GLOBAL VARIABLE FOLLOWING! | SIZE:int
int test_global;
//ATTENTION: GLOBAL VARIABLE FOLLOWING! | SIZE:int
int secured_i;
//ATTENTION: GLOBAL VARIABLE FOLLOWING! | SIZE:long
long secured_sum;
….
```

You can see the annotations with the sizes. These help a python script that runs afterwards to initialize the proper number of bytes.

How the stack is secured:

The secure stack is an array allocated in the normal, “unsecure” heap. We use two global variables which function as registers (base pointer and stack pointer), and, starting the stack pointer from the end of the stack, we decrease it whenever we need to allocate something into the secure stack (and increase when we deallocate).
Like the globals, we interleave keyshares+MAC bytes among the useful bytes, and we use specific getters/setters for accessing the useful bytes (that pay attention to fetch/set the correct bytes and not the keyshare ones/MAC ones , as well as to verify/update the MAC when needed).
In this secure stack we implement our own calling convention, similar to the ones that are used in real applications. When we encounter a “function call”, we call the stack allocator to allocate a function frame (with the size we want), we call the setters to set the function arguments, return address, base pointer, possibly stack canary, and local variables that need initialization. The jump to the function code (and out of it) is done with a goto. In order for this to work in C, the code of all our tests is in one big function, great_function_that_wraps_the_tests() , in **template_files/ tests_with_new_stack_template.c .** We need to do that because we cannot jump to a label that is outside our function.

Here’s some code that uses the secure getters/setters of some variables that reside in our stack (note that their names are previously defined as an offset from the base pointer):

```
 for ((int)SET_STACK_INT( i , 1); (int)GET_STACK_INT( i ) <= 10; (int)SET_STACK_INT( i ,
(int)GET_STACK_INT( i )+1))
 {
        (double)set_stack_double_array_element( min_heap , (int)GET_STACK_INT( i ) , 10 -
(int)GET_STACK_INT( i ));
      (int)set_stack_int_array_element( min_heap_corresponding_indexes , (int)GET_STACK_INT( i ) ,
(int)GET_STACK_INT( i ));
              (int)set_stack_int_array_element( fixed_indexes , (int)GET_STACK_INT( i ) ,
(int)GET_STACK_INT( i ));
 }
```

Also, here’s some code that calls a function:

```
({
//allocate space
void *returned_addr_after_allocating_1002=allocate_mem_into_secure_stack_in_chunks(8);
if (returned_addr_after_allocating_1002==NULL)
        {printf("ERROR! no stack mem left -> line %d\n",__LINE__);exit(8);}
//set return address
set_stack_pointer_array_element(returned_addr_after_allocating_1002-
(3+1+1)*(stack_bytes_used_for_keyshares+number_of_mac_bytes+stack_bytes_for_useful_data),1,
```

```
&&return_label_mh_combine_no_3);
//create a normal array that will be inserted into the secure stack, and will hold the arguments
long size_of_array_for_array_fun_parameters_1003=2*4;
int array_for_int_fun_mh_combine_params_1003[2];
array_for_int_fun_mh_combine_params_1003[0]=(int)GET_STACK_INT( new_ind );
array_for_int_fun_mh_combine_params_1003[1]= (int)GET_STACK_INT( num_of_elements );
//put that array in the secure stack
insert_data_into_stack_mem(size_of_array_for_array_fun_parameters_1003,(unsigned char*)
array_for_int_fun_mh_combine_params_1003, (unsigned char*)returned_addr_after_allocating_1002-(3-0)*
(stack_bytes_used_for_keyshares+number_of_mac_bytes+stack_bytes_for_useful_data)); long
size_of_array_for_array_fun_parameters_1004=3*8;
//do the same for the pointer parameters
void * array_for_pointer_fun_mh_combine_params_1004[3];
array_for_pointer_fun_mh_combine_params_1004[0]=(double*)GET_STACK_PTR( min_heap );
array_for_pointer_fun_mh_combine_params_1004[1]= (int*)GET_STACK_PTR( corresponding_indexes );
array_for_pointer_fun_mh_combine_params_1004[2]= (int*)GET_STACK_PTR( fixed_indexes );
insert_data_into_stack_mem(size_of_array_for_array_fun_parameters_1004,(unsigned
char*)array_for_pointer_fun_mh_combine_params_1004,(unsigned char*)
returned_addr_after_allocating_1002-  (3-1)* (stack_bytes_used_for_keyshares +number_of_mac_bytes
+stack_bytes_for_useful_data));
//set the base pointer in the stack to hold the former value of the "register"
set_stack_pointer_array_element(returned_addr_after_allocating_1002- (3+1+1)*
(stack_bytes_used_for_keyshares+ number_of_mac_bytes+ stack_bytes_for_useful_data), 0,
base_pointer_for_stack);
//set a new value for the "register"
base_pointer_for_stack=returned_addr_after_allocating_1002-(3+1+1) *(stack_bytes_used_for_keyshares+
number_of_mac_bytes+ stack_bytes_for_useful_data);
//goto the function code (the locals are set in there)
goto mh_combine_start_label; return_label_mh_combine_no_3:  ;
//get the return value and use it as the return value of this big block of code
get_stack_int((last_unused_stack_memory-
(3+1)*(stack_bytes_used_for_keyshares+number_of_mac_bytes+stack_bytes_for_useful_data)));
});
```

---- continue from here:

Here's a simple overview of the directory structure:

Directory structure:

1) src/: Inside there are the java source files than manipulate the assembly and the final executable

2) code/: In there are the other files that emulate the secure machine. There's a lot of stuff there. In the **template_files/** we have the templates of the code that are copied and changed accordingly (as the configuration of the compilation demands). In **pycparser_scripts/** we have some cool python code that transforms C code → C code that uses the secure structures (does not support the full C spec but it's still cool). In **scripts_for_code_generation/** we have some other scripts that help for the emulation of the secure system. Th other directories are not THAT important.

3) bin/: This directory does not exist on github. In there there are the java class files (that Eclipse produces automatically). Someone that does not use Eclipse should compile the .java files in the src/ directory and move them inside the bin/ directory. To save the hassle, a script does it automatically, if the java compiler (javac) is

installed.

Some constants of our implementation.

<u>Description of the most significant files:</u>

First, we have a .c file that contains ALL of the code (except the functions of the standard C library that it #includes). That code is spread among different C files which are included by that main file. That code includes the main_program_function() function (that's practically the main, except some crypto initializations, so the actual main() has to be somewhere else), test functions, the secure heap implementation, the secure stack implementation and the verification procedure.
That program is compiled in its final "secure" form after some changes are done to it. So we actually work with the template of that program, to which we make changes and then compile.

Let's now talk about the significant files.

Let's open **code/template_files/** directory and take a look at **main_program_template.c.** It references the secure globals declaration in headers_needed_template.h (take a look at the python calls), we can see that it includes a file called "functions_needed_header.c". After that there is the main_program_function() and then another file is included, the "functions_needed_footer.c". These two files include every other function that may be needed. If we want to peek inside, we have to open **functions_needed_header_template.c** and **functions_needed_footer_template.c** .
That is not necessary. What needs to be understood are two things: a) Inside our template files there are calls to python scripts that change the code, so the final program that is compiled is the output of the python scripts (which pad the templates with proper C code), b) Everything that is to be secured is in one compilation unit (since we use all those includes), c) The template files are copied (often with some changes) to the non-template equivalents, and the compilation is done in the latter. The code that a programmer will write should be in the templates though.

Now let's delve deeper into the securing procedure, explaining it step by step. The core of that procedure is a bash script, **code/automate.sh** . What it does in summary: Takes the user parameters, changes the templates using python, compiles the program into assembly, pads NOPS among the assembly commands (more on that later), assembles into executable, and changes the NOPS with keys and macs inside the executable.

Let's open it and see it. In the start we can see the parameter fetching (and setting), some error checking and compilation of the java files if they are out of date. Then we see a line which has the string "python3 **set_correct_defines.py**". Let's go ahead and

open that python file, inside **scripts_for_code_generation/** directory. It consists of two parts. The first one sets the proper parameters in the file “headers_needed.h”, and the second one is obsolete (is used for the former stack implementation, so we don't have to pay special attention to it). The file **template_files/headers_needed_template.h** is a header file included by almost every file in the project. It has many compile time parameters (like the size of the heap and the stack, the number of the keys etc), some typedefs and the names/types of the global variables. As we can see, the python script alters some of these parameters, as the user requests. It writes to the file “headers_needed.h” inside the **code/** directory. After that, every unit includes the non-template file “headers_needed.h” which has the proper parameters.

---OLD STUFF BELOW ---

Let's move on inside the bash script now. We see that some template files are copied into their non-template counterparts. These ones contain the heap manager functions, and the tests that we run. The content of the two files is then put into the third one (using “sed” command). The last file, containing all the tests, is the **tests_with_new_stack.c**. That file, however, has not taken its final form yet. In order to see what has been put to be tested, open **template_files/more_tests_that_use_new_stack_template.c** . You can see that there is C code, but there are some parts that ask python to call a function. That code part is later replaced by python to contain valid C code.

Let’s continue in automate.sh:

We see that the file **insert_new_stack_commands.py** is called (or its sister file, the **insert_new_stack_commands_groups_of_vars_as_arrays.py**). These are files that pad the C code that manipulates the secure stack. For example the file **tests_with_new_stack.c** (which is copied from is corresponding template **template_files/tests_with_new_stack_template.c)** makes a call “ //HEY PYTHON CALLING FUNCTION : sieve_of_Eratosthenes_sec | PARAMETERS TO CALL WITH: 20000000 ” (the same way as seen in the previous .c file). That call is parsed by python and replaced with the proper code that calls that secure function. The code that is replaced can be seen in the non-template file inside **code/** directory. If we open “tests_with_new_stack.c” we see that much code has been padded by the python script. Now, this file is in its “secure” form and it is ready to be included by the main program.

Moving on inside automate.sh, we see that some template files are copied inside **code/** directory. These files do not have any code that needs to be altered, and are simply copied from their template form to their usable form.

The next step is to create the structure of the globals struct in **headers_needed.h** . The python script that it is used, (**insert_keys_and_macs_among_globals.py**), but at this time it only changes the structure of the globals, it does not touch the other files it references inside. That is achieved with the flag

“DO_STUFF_FOR_HEADERS_NEEDED_ONLY”.

Next, some compilation takes place. Some files (like the implementation of the MAC, the secure getters and setters from the memory, the crypto initialization functions, stack allocation and deallocation etc), do not need to be secured. In fact, we assume that they are implemented in the hardware inside our secure CPU. So, their code should be a) Not secured, b) Optimized as much as possible to get close to their hardware implementation. We create their object files, in order for them to be able to be linked along with the secure code.

The next step is to initialize the global variables. Inside the **template_files/headers_needed_template.h** we can see some global declaration. This declaration has been padded with keys and macs before (in the **headers_needed.h** file). Inside the **template_files/main_program_template.c** (at the start), we can see the placeholder for their initialization. So, the python script **insert_keys_and_macs_among_globals.py** (remember: it’s the second time this script is called, but now with a directive to touch all the files). same does what its name says. We can see its output if we open the **code/main_program.c** and look at its start. Among the globals there are the initialization values of the globals and their keys+macs.
Note: we recreate headers_needed.h for reasons of correctness.

After that we have some more cp’s ,secure getters and setters compilation, and the main compilation of the program. To see what happens there, see the **Makefile**. It compiles the main program (with all of its #includes) into an assembly file, and then it compiles the assembly file into an executable, which serves as the baseline for our benchmarks. That executable contains code that does not use the secure getters/setters and its code is not secured, so it can give us benchmarks for the normal execution time.

However, after that stage we have an assembly file that some parts of it (which are to be benchmarked) use secure operations. It looks like this:

Άνοιγμα

main_program.s
~/virus_detection/gcc-Linux-Implementation/code

Αρχείο Επεξεργασία Προβολή Αναζήτηση Εργαλεία Έγγραφα Βοήθεια

```
    movq    16(%rax), %rax
    testq   %rax, %rax
    je  .L354
    movl    $541, %edx
    movl    $__func__.13974, %esi
    movl    $128, %edi
    call    error_checking_malloc
    movq    %rax, %rdx
    movq    -64(%rbp), %rax
    movq    %rdx, 16(%rax)
    jmp .L355
.L354:
    movq    -64(%rbp), %rax
    movq    $0, 16(%rax)
    movl    $.LC87, %edi
    call    puts
.L355:
    movq    -88(%rbp), %rax
    movq    16(%rax), %rax
    testq   %rax, %rax
    je  .L356
    movq    -64(%rbp), %rax
    movq    16(%rax), %rax
    movq    -88(%rbp), %rdx
    movq    16(%rdx), %rdx
```

Now we approach the final stages of the automate.sh . The next step is to secure the assembly code. That is the job of the first java program. It (**Secure_Assembly_new.java** in **src/** directory) a) changes that assembly into an assembly that has NOP instructions inserted among groups of regular instructions. For example, if 7 NOPs are inserted between groups of 1 regular instruction, the new assembly file looks like: (Note: the picture is old, now the name is main_program_sec.s and code for verification is added). Notice the jmps above the NOPs.

```
set_correct_defines.py ×   Helloworldadd.s ×   Helloworldadd_sec.s ×
NOP
NOP
.UNIQUE8537:
    call    set_stack_int_array_element
.L679:
 jmp .UNIQUE8538
NOP
NOP
NOP
NOP
NOP
NOP
NOP
.UNIQUE8538:
    movq    -24(%rbp), %rax
 jmp .UNIQUE8539
NOP
NOP
NOP
NOP
NOP
NOP
NOP
.UNIQUE8539:
    movq    16(%rax), %rax
```

The above picture shows the idea, but the current implementation uses the following numbers:
The bytes that will hold the normal instructions are a multiple of 16. If the commands do not fit exactly inside such a block, NOPs are padded (after the jmps) to make them (useful bytes + padded NOPs) a multiple of 16 (more on that later).
The keyshares are 16+16=32 bytes.
The MAC bytes are 16 bytes.
So, in total, one will see many more NOPs between the useful bytes.

Apart from that, b)this program adds code for on the fly verification of the code MACs at the start of each block, under the idea that the verification code is invoked as many times as a secure CPU would check the MACs of the code.

In our execution of the automate.sh script, we have, apart from 32 nops for keys and 16 nops for macs , some more nops for control, in which we store data that help us to identify what is what when we parse the bytes (e.g. canaries). So, the final assembly looks like (the nops are many): (Note: the first 3 lines [pushfq...popfq] are for on-the-fly code verification, they are not added by the compiler).

Άνοιγμα ▾

main_program_sec.s
~/virus_detection/gcc-Linux-Implementation/code

Αρχείο Επεξεργασία Προβολή Αναζήτηση Εργαλεία Έγγραφα Βοήθεια

```
NOP
NOP
NOP
NOP
.UNIQUE2:
pushfq

call do_verify_code_on_the_fly
popfq
    movq    %rax, -8(%rbp)
    cmpq    $0, -8(%rbp)
    jne .L4
    movq    stderr(%rip), %rax
    movl    -36(%rbp), %ecx
    movq    -32(%rbp), %rdx
    movl    $.LC0, %esi
 jmp .UNIQUE3
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
```

Now comes the part of adding the padded NOPs. Since x86 commands are of variable size, and our model dictates that the code blocks are fixed size, we need to pad the code blocks with enough bytes (we use NOPs), so that we reach a certain size. The NOPs are padded after the jmps (they are not executed). Our goal is to reach a code block of the following structure:

<verification_code> || <useful_code_bytes> || <jmp_to_next_block> || <canaries_and_bytes_for_control> || <padded_NOPs> || <keys> || <MAC>

The sizes are the following:
verification code: 7 bytes
useful_code_bytes: variable size
jmp_to_next_block: 5 bytes
canaries_and_bytes_for_control: 3 bytes for canary bytes, and 2 bytes for control (they contain in a 2-byte int the size of the bytes from the verification code until the jmp)
padded_NOPs: variable size, so that the entire block is a multiple of 16
keys: 32 bytes (16 bytes = 128 bits for each key)

MAC: 16 bytes

The padding of the bytes is done in three rounds:
First we assemble and disassemble the entire program (to find the size of the commands).
Then, a python program (**assembly_parser_and_size_finder.py**) goes through the disassembled output (output of objdump) and finds the sizes of the commands. It groups the NOPs that it finds together (only if they are more than 10 together). One can see the output of this program in the file **assembly_sizes.txt**
Secondly, another program (**nop_padder_for_fixed_size_code_blocks.py**) parses both the disassembly output and the sizes of the commands, and decides how many NOPs to pad.
The third and final round is the verification that everything has been done properly (that is done by the program **assembly_fixed_size_verifier.py**).

Now we have an assembly file with padded NOPs after every X bytes instructions (note: some instructions-calls and jmps- force creation of new blocks), and before these X bytes of instructions we have the verification code. The next part of the automate.sh script assembles that code and produces the executable (the “make secure” part).

------ OLD, WRONG STUFF BELOW -------
The only thing that is left is to change the NOPs with the keys and macs that should be among the code lines (the control bytes should be changed too). That is the job of **Secure_Machine_Code_new.java** in **src/** directory. After it is finished, we have the final, secure executable.

How to run a simple compilation

Let's run a simple compilation together. First we have to define what will be run. Please open **template_files/main_program_template.c** . There, in the main_program_function(), you can see a lot of commented code (that runs benchmarks). The lines that calculate the primes up to 100 are just for correctness testing (and use the old and obsolete way of function calling which does not implement a good simulation), so if you want you can comment them out.
We are interested in the function great_function_that_wraps_the_tests(), which engulfs all the secure functions and manages to call them in a “secure” way, through the secure stack. That function is in **template_files/ tests_with_new_stack_template.c** and as you can see, at the first 500 or so lines has the secure functions definitions and their code. Each function footprint is described at the top with the appropriate annotations, and then its code follows.
After the functions, there is the code that calls them. An annotation starting with

“HEY PYTHON CALLING FUNCTION” means that at that point the secure function will be called with the specified parameters.
All the secure functions are inside the same C function, because in the secure stack there are no calls, just gotos and we have to know the address to which we have to jump (so it must be within the same function).
So, uncomment any function that you want to run. For example, uncomment the part that calculates the primes, and set the parameter to 1000 (i.e. calculate the primes up to 1000).
So it's time to compile and then run the secure executable. Compile it with “./automate.sh 32 30 3 64 250000 64 250000 64 16 73” and wait for the script to finish. If it is the first time you are running it, it might take some time to complete because it is building a table with assembly command sizes. First of all, what the parameters are about:
The 32 is the number of keyshares, in bytes. There are 2 keys in the memory, and each is 16 bytes (128 bits) long. A user should not change this.
The 30 is the (maximum) number of assembly commands that each code block will have. A code block may be split should we encounter a label or a function call, since when dropping (jumping) onto the label or returning from the function call, we have to verify a mac. So we put labels at the start of blocks, and calls at the end (and when jumping to that label or returning from the function call, the mac verification is executed exactly afterwards).
The 3 is the number of canaries which follow the jmp above the keyshare, so as to make sure that that jmp is not some kind of another jmp that belongs to the normal code.
The 64s are the number of the useful bytes that reside in each block of the secure heap,stack and data segment (in that order). So every block in the heap for example, will contain 64 useful bytes, 32 keyshare bytes and mac bytes (16 in the above example).
The 250000s are the sizes of the secure heap and stack (in that order). They include the keys and macs. If they are not enough to host a whole number of blocks, they are being expanded a bit to do so.
The 16 is the number of mac bytes (128 bits), and usually this should not change by the user (unless he knows what he's doing).
The 73 is the number of useful bytes that a code block will have. When there is a parameter after the number of the mac bytes, this means that we want a fixed size block of code. So, now every block will have up to 73 bytes of code (for fixed size blocks the on-the-fly verification call is 7 bytes and the jmp 2, so the actual useful bytes are 64 – here equal to the data blocks). When we are forced to split a block or the assembly commands don't add up exactly, we pad NOPs after the jmp so that the distance between two blocks remains fixed.
So a block in the code would be:
<verification call> <useful bytes> <jmp> <canaries> <1 byte> <padded NOPs> <keyshares> <macs>
Note: After the canaries we use 1 byte to denote the length of the actual code before the canaries. After that byte the padded NOPs follow.

The verification call, jmp, canaries and that 1 byte will not be there, should specialized hardware is used.
Due to implementation restrictions (we want the jmp above the keyshares to be 2 bytes long), the maximum number of code bytes for a block is 86 bytes. That should be enough as for bigger sizes more complex hardware support is needed (we need atomic fetching of the bytes from the memory).
If we omit the 73 at the end, the code blocks will be of variable size, and the limit will be the 30 commands (except if we need to split blocks). Then, the verification call costs 14 bytes, as the size of the block has to be known as well.

Let's open **automate.sh** to see what else can be configured before the compilation. There are many more parameters that can be altered by editing the automate.sh script and recompiling.

The first parameters that we encounter are
SECURE_GLOBAL_VARIABLES_WITH_SEPARATE_KEYS=1 #default 1
DECLARE_GLOBAL_KEYS_AS_AN_ARRAY=0 #default 0
INSERT_PARAMERERS_INTO_NEW_SECURE_STACK_AS_ARRAYS=1 #default 1

The first two affect the way that the global keys are made. Whether they are 0 or 1 there is not much difference. The third one however should be 1 for performance reasons, as it does not waste space in the secure stack. When variables of the same type (ints, chars, etc) are function parameters or local variables to a function, they are placed close to each other (as in an array), and we don't allocate an entire block for each one of them.

The parameter ADD_CODE_ON_THE_FLY_VERIFICATION is about adding a verification call at every block access (at the start of each code block and at read and write of data blocks). It should be 1.

FORCE_NUM_OF_INSTRUCTIONS_OVER_NUM_OF_BYTES: If it is 1, raises an error if the number of commands per code block is not satisfied with every code block (except when we split blocks). So, if we request 50 bytes per code block and 30 instructions, if there is a block in which 30 instructions don't fit (highly likely), an error is raised. Default is 0.

DO_NOT_MAC_WHAT_WHE_ADD_IN_CODE: This option (default 1) does not include in the mac calculation for the code blocks the verification code, the jmp, the canaries and the byte after them. The padded nops are included. That is to approach the performance of the actual hardware that would not have all these extra bytes there.

USE_INLINE_CODE_FOR_DELAYS: This option should only be activated when the user tries to benchmark normal times. The normal functions are in **template_files/**

general_tests_template.c and Hanoi is at **template_files/ stack_manager_test_suite_template.c** . We have added some extra code before and after every (recursive) function call, because we have to be fair to our calling convention, which, as it is not implemented in hardware, has to do some extra things before and after actually calling the secure function (like initializing the function frame).

CODE/DATA_CACHE_TYPE: In order to speed up performance, the user can set a secure cache size. In that cache, every recent block of code/data is put (when reading or writing to it), so the mac will not have to be calculated if it is accessed again. These parameters set the cache type, 0 for fully assosiative, 1 for direct mapped, 2 for set assosiative cache. Default is 2.

NUM_OF_CACHED_BLOCKS_OF_CODE/DATA: The cache size, in blocks.

CODE/DATA_CACHE_SET_ASSOSIATIVE_SIZE: Should the cache is X-way set assosiative, these parameters determine the X. Default is 2.

IGNORE_MACS_EVEN_IF_THERE_ARE_MAC_BYTES: This parameter tells the compilation procedure to put the macs in the memory, but do absolutely nothing about them (i.e does not add verification code if a block is accessed, and does not split code blocks). Default 0.

IGNORE_MACS_LAST_MOMENT_EVEN_IF_THERE_ARE_MAC_BYTES: This parameter puts the macs in memory, puts the on-the-fly verification code, but does nothing when it comes to the actual mac calculation. Default 0.

FORCE_CODE_BLOCK_SPLIT_ON_LABELS_AND_CALLS: This parameter should only be 1 when IGNORE_MACS_EVEN_IF_THERE_ARE_MAC_BYTES is 1 too. It splits the code blocks, should a label or a call command is encountered. Default 0.

TREAT_LOOP_COUNTERS_AS_UNSECURED_VARIABLES: This parameter was made under the following idea: The loop counter variable in a loop usually resides in a register and the memory is not altered, so we should not secure it (calculate the macs when it changes) in the secure program. Turns out that without optimizations the normal program seems to touch the memory for every change in the loop counter, so the loop counters have to be secured, too. Default 0.

SQEEZE_KEYS_WHEN_MACING: This parameter implements an optimization in mac calculation. The keys are 32 bytes long (both), but we don't need all that randomness for security when calculating the mac.We can XOR every two bytes together and have 16 bytes for keys, when macing. In that way we have to calculate one less 16-byte block for the mac. The security does not drop under a dangerous threshold. Default 1.

COUNT_MAC_INVOCATIONS: When this parameter is 1, the number of mac invocations is calculated and presented at the end of the program. Since that's what we care about when we activate it, the mac (for efficiency reasons) is not properly calculated, but is replaced with 0's. This is the parameter that was active in the benchmarks, in order to calculate the total extra mac time. Default 0.

ADD_THE_PADDED_NOPS_IN_THE_MAC_IN_FIXED_SIZE: This parameter takes into account the padded NOPs for every block of code (if we use fixed-size blocks) for the mac calculation. Default 1.

USE_CODE_CACHE_WITH_UNSPLIT_BLOCKS: This is an experimental parameter that can throw errors. It tries to guess in which code block a certain assembly command would be, if we did not enforce code block split into the secure program (as the secure hardware would do – it would not split blocks). Then what is and what is not in the cache is calculated relative to the unsplit blocks' addresses. The performance increase, for large code caches, is marginal. Default 0.

SET_AS_GIVEN_THAT_EVERYTHING_MACED_WILL_BE_FIXED_AND_MULTIPLE_OF_16: This parameter tells the compilation script that the code block size and the data block size will be the same size, and the useful bytes will be a multiple of 16 (like in “./automate.sh 32 30 3 64 250000 64 250000 64 16 73”). In that case, we can safely ignore length prepending when we mac, as well as padding. That makes the mac calculation a bit faster.

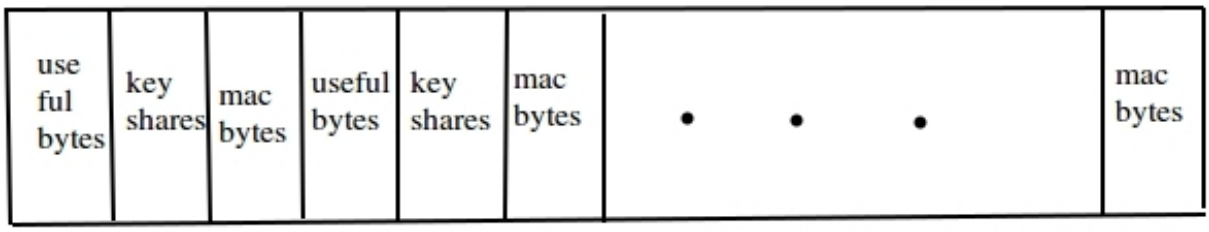
These were the extra options, and now you can run the automate.sh with any parameter you want, to compile again. There are 2 programs produced:

./main_program: This is the unsecured version. Will throw errors in the end because it will try to do some secure stuff, but one can measure normal time executions here.

./main_program_ksec: This is the secure program. You can run it to see the secured code execute.

How secure memory is implemented

This section describes the way we implement the memory structure in the data segment, the heap and the stack (not the code segment). As the paper suggests, we interleave keys and macs among the useful bytes in memory. A picture is worth a thousand words:



The mac bytes contain the mac of the concatenation of <useful bytes>||<keyshares>.
As far as sizes are concerned, the keyshares are 32 bytes (2x 128 bit keys). The mac bytes are the output of AES-CBC-MAC. So, as the last block is 16 bytes (128 bits), the size of the mac is 16 bytes too. The useful bytes are variable. However, there are some things to consider.
a)When we verify the memory on the fly, we need to calculate as few CBC-MAC rounds as possible, so the less blocks we calculate the best. If we need 1 char (1 byte) from a useful byte area and the useful byte group is 20 bytes (or, more generally>16 bytes), then the mac will have to be calculated for about ceiling((20+32) div 16) =4 rounds. And all that for just 1 character. However, if the block is <=16 bytes, then we do the job with 3 rounds.
b)The secure CBC-MAC implementation requires adding the length of the data to be maced in the front, so we use 1 byte for that. Also, libcrypto's implementation of CBC-MAC uses 1 extra block for padding if it has to mac bytes whose length is multiple of 16. So, to achieve mac calculation with 3 blocks, we need the useful bytes to be maximum 14.
c) We use a specific way of storing variables into the secure memory. If we have to store an integer (4 bytes), we use an entire useful bytes block for it. Same for every other variable(chars, longs, etc), except for arrays. We allocate just enough useful byte groups to store the array, and its elements are one after another in the useful byte groups. So, if the useful bytes length is 6 bytes and we have an array of 2 integers, the first integer will be stored in the first useful byte block, and the second one will be split between the first and second useful byte groups:

<int1_byte1><<int1_byte2><int1_byte3><int1_byte4><int2_byte1><int2_byte2><keys><mac> <int2_byte3><int2_byte4><byte><byte><keys><mac>.

So, in case we need to fetch the second integer, we have to verify both the first and the second mac, which adds a lot to the delay.
That leads to the conlusion that 14 useful bytes may cause longs and pointers (8 bytes in 64 bit architecture) to be split between useful byte blocks. So, the optimal size of the useful bytes group is 8 bytes long, which causes no splits.
Update: This is the optimal size indeed (for no secure cache), but we have to have different size for code blocks and data blocks. The reasonable approach is to make them the same size. Should the user set it like this (in **automate.sh** and in the command-line parameters), and it's a multiple of 16, there is the option to disable padding and length prepending, so the overall MAC calculation becomes a bit faster. There is worse performance for useful bytes>8 if we use no secure cache, but not if we do use. If we use, larger useful blocks go into the cache and we have to calculate less macs as a result.

The way to access the secure memory is through secure getters and setters. Functions like “void set_stack_int(void * start_of_secure_data,int source)” are used to store data to the secure stack, heap or the secure globals. We have implemented secure getters/setters for a subset of the entire C functionality (our subset is still strong enough to be able to describe most common problems). More specifically, we have getters and setters for chars,ints,long ints, floats,doubles, pointers and their arrays. Furthermore, we have a way to store an arbitrarily-sized data structure into the secure memory, which is fetched from the unsecured memory (and of course the inverse, fetched from the secure memory and stored into the unsecured).

How the globals are secured:

The secure global variables all reside in a struct. Please open **template_files/ main_program_template.c** . At its start the global declaration is obvious. When we need to declare a global, we first add a directive to the python scripts and then denote the size of the global. Python parses our commands and then initializes the variables, adding keys and macs after them in the next struct (which is empty in the template). Please open “main_program.c” in “code” directory (after compiling using the automate.sh script). If we take a look inside it, we can see the globals,the keys and the macs, and after them their initial values.
The secure getters and setters are straightforward (see the last lines of **template_files/secure_getters_setters_template.h**). We have the standard type of getters, and generic setters for normal blocks of code and for for loops. You can see one example of secure global access in the “void secure_global_sum_calculator(int times, int maxval)” function inside **template_files/general_tests_template.c.**
The way we do this right now, every block of <useful_bytes>||<keys>||<macs> can only hold only one global variable. Should someone want to use arrays of variables, he has to use the heap.

How the heap is implemented

The heap is allocated in order to have space for the keys and macs. As soon as it is allocated, the keys are populated from the file **heap_keyshares** in **code/** directory. That file is generated by Secure_Machine_Code.java. After that, the macs are calculated and put in the secure heap as well. The implementation of the heap is done in **template_files/memory_manager_template.c** and the getters and the setters in **template_files/secure_getters_setters_template.c.**
A user can allocate and deallocate chunks of memory in the heap, and we keep track of them with two doubly-linked lists, one for the free chunks and one for the allocated ones. The standard way to do that is managed_secure_malloc() and managed_secure_free().

How the stack is implemented

The stack has a similar memory image and initialization phase as the heap. The stack keyshares are read from **stack_keyshares** file and the macs are calculated on the fly.
The code that implements the secure stack is in **template_files/ secure_stack_manipulation_functions_template.c** and the secure getters and setters in **template_files/ secure_getters_setters_template.c** and in **template_files/ secure_getters_setters_template.h.**
The way to allocate in the secure stack is to move the stack pointer, which is represented by a global variable. The function to allocate is allocate_mem_into_secure_stack() and the function to free is free_mem_from_secure_stack() .
The stack is used to hold the function frames. Since we have to use the secure stack to call the functions, we have to implement a calling convention by using the secure stack.
Every function frame has:
a) the arguments of the function (1 ... n) in that order (so functions of variable arguments are not supported)
b) the return value that the function may return (if it does return something)
c) the return address to where we should jump when we finish the function execution
d) the base pointer, which serves as an anchor relative to which we can access all the function frame elements
e) the local variables of the function.

The way the code is padded can be seen in **code/ insert_new_stack_commands_groups_of_vars_as_arrays.py** and in **code/ insert_new_stack_commands.py** (they do the same thing, they simply set the

function arguments and local vars in a different way).

After you have run the automate.sh script once (for example “./automate.sh 32 30 3 64 250000 64 250000 64 16 73”), you can see a file tests_with_new_stack.c in **code/** directory. If you open it you can see that for each function declaration,implementation and call, many new lines have been added. For every function:
At first there are some #defines, which give names to offsets from the base pointer for every stack frame. They are there to name the local variables or the parameters of the function, so as to make the code that has to be written more user friendly.
At the return point of a function code there is the code that frees the stack frame, #undefines the formerly defined names and jumps to the return address.
When a function is being called, there is a lot of code that is being added. The space in the secure stack is allocated, the parameters and the local variables are initialized (as well as the return address and base pointer), and then there is a jump to the function code. When the function returns, the return value is set and the execution continues.
That is a simple way of calling functions, and no optimizations were implemented for simplicity reasons.